

**Московский авиационный институт (национальный
исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Курсовой проект по курсу «Программная инженерия»

Студент:	О.А. Баранов
Преподаватель:	А. А. Кухтичев
Группа:	М8О-111М-23
Дата:	01.05.2025
Оценка:	4
Подпись:	

Москва, 2025

Задание курсового проекта

Задача: Требуется разработать современное веб-приложение. Основные требования:

- Поднять веб-сервер (nginx). Отдавать статические файлы (логотип и т.д.) по location'у /static/. Настроить проксирование запросов на сервер-приложений по отдельному URL;
- В конфиге nginx создать location, которое будет ходить на FastAPI приложение;
- Поднять сервер-приложений;
- Создать базу данных в PostgreSQL; нанисать классы-модели, мигрировать;
- Организовать приём и передачу сообщений с помощью формата JSON, используя REST;
- Реализовать метод API для загрузки файла, использовать для хранения файла облачное S3 хранилище, создать location в Nginx для раздачи загруженных файлов, реализовать обработчик в приложении для проверки прав доступа к файлу;
- Реализовать OAuth2-авторизацию для двух любых социальных сетей, навесить декоратор, проверяющий авторизацию при вызовах API;
- Покрыть тестами все вьюхи и по желанию другие функции; Написать seleniumтест (найти элемент + клик на элемент); Использовать mock-объект при тестировании; Использовать factory boy; Узнать степень покрытия тестами с помощью библиотеки coverage;
- Развернуть и наполнить тестовыми данными Elasticsearch; Реализовать поиск по пользователям, продуктам (сущностям); Реализовать метод API для поиска по указанным сущностям и создать страничку HTML с вёрсткой для поиска и отображения результатов;
- Установить и поднять centrifugo; Подключить centrifugo к проекту на стороне клиента и сервера; Организовать отправку/получение сообщений с помощью centrifugo;
- Создание docker-compose для нпроекта:
 - nginx;
 - База данных;
 - FastAPI-приложение.

Создание Makefile для проекта.

Тема курсовой: Аналитический портал

Вариант веб-сервера: nginx.

Вариант сервера-приложений: FastAPI.

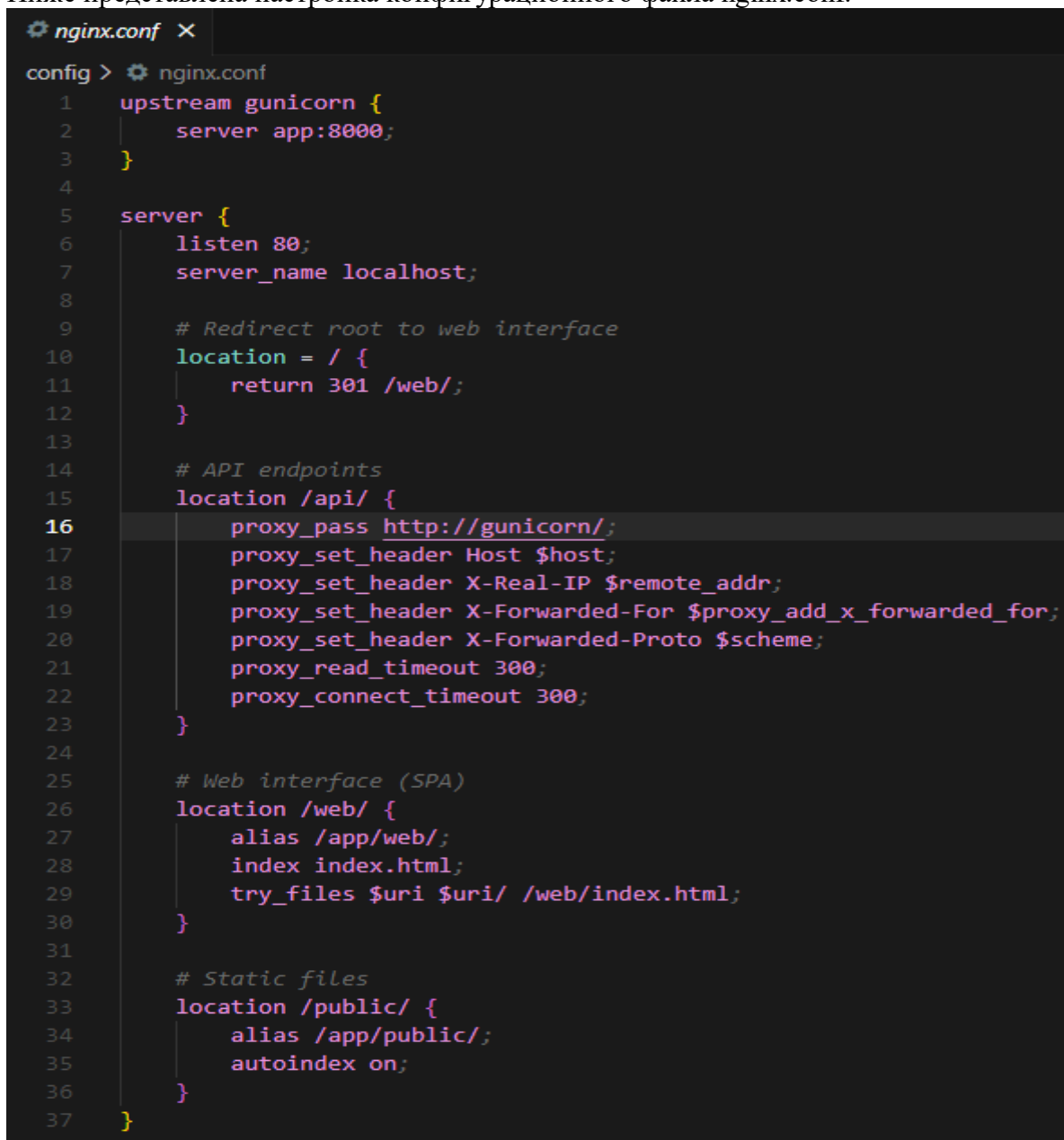
Вариант базы данных: PostgreSQL.

1 Веб-сервер

В качестве веб-сервера в курсовой работе был выбран Nginx [1]. Nginx - это программное обеспечение с открытым исходным кодом, которое используется для создания веб-сервера. Я использовал его как сервер для статических файлов, а также в качестве обратного прокси-сервера. Обратный прокси-сервер получает запросы от клиентов и передаёт их на серверы.

Для настройки веб-сервера я создал конфигурационный файл nginx.conf. Основной задачей сервера является обеспечение оптимальных условий для передачи данных между клиентами и серверами, что позволяет эффективно обрабатывать запросы.

Ниже представлена настройка конфигурационного файла nginx.conf:



```
config > nginx.conf
1  upstream gunicorn {
2      server app:8000;
3  }
4
5  server {
6      listen 80;
7      server_name localhost;
8
9      # Redirect root to web interface
10     location = / {
11         return 301 /web/;
12     }
13
14     # API endpoints
15     location /api/ {
16         proxy_pass http://gunicorn/;
17         proxy_set_header Host $host;
18         proxy_set_header X-Real-IP $remote_addr;
19         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
20         proxy_set_header X-Forwarded-Proto $scheme;
21         proxy_read_timeout 300;
22         proxy_connect_timeout 300;
23     }
24
25     # Web interface (SPA)
26     location /web/ {
27         alias /app/web/;
28         index index.html;
29         try_files $uri $uri/ /web/index.html;
30     }
31
32     # Static files
33     location /public/ {
34         alias /app/public/;
35         autoindex on;
36     }
37 }
```

Рисунок 1. Файл nginx.conf

Конфигурация Nginx настраивает веб-сервер для обработки запросов на localhost через порт 80. Основные компоненты конфигурации:

1. Upstream `gunicorn` определяет сервер приложения (Django) на порту 8000 с именем `app`, что соответствует Docker-сервису.

2. Корневой запрос (``) перенаправляется на веб-интерфейс через:

```
location = / {  
    return 301 /web/;  
}
```

Это означает, что при обращении к <http://localhost> будет выполнено перенаправление на <http://localhost/web/>

3. Для API-эндпоинтов создан блок:

```
location /api/ {  
    proxy_pass http://unicorn/;  
    ...  
}
```

Директива `proxy_pass` перенаправляет все запросы, начинающиеся с `/api/`, на Unicorn (FastAPI-приложение). Дополнительные заголовки (`X-Real-IP`, `X-Forwarded-For` и др.) обеспечивают корректную передачу информации о клиенте.

4. Веб-интерфейс (SPA) обслуживается через:

```
location /web/ {  
    alias /app/web/;  
    ...  
}
```

5. Статические файлы доступны по:

```
location /public/ {  
    alias /app/public/;  
    autoindex on;  
}
```

Запрос к <http://localhost/public/1.jpg> будет искать файл по пути `/app/public/1.jpg`. Включенный `autoindex` позволяет просматривать содержимое директорий.

Все пути (`/web/`, `/api/`, `/public/`) обрабатываются относительно разных корневых директорий, что обеспечивает четкое разделение между статическими файлами, API и веб-интерфейсом.

2 Сервер-приложений

Серверная часть реализована на современном Python-фреймворке FastAPI [2], который обеспечивает высокую производительность и удобство разработки RESTful API. Основные компоненты сервера:

1. Инициализация приложения

Приложение создается с базовыми метаданными:

```
app = FastAPI(  
    title="MAI Backend API",  
    description="API для блог-платформы",  
    version="1.0.0",  
    root_path="/api"  
)
```

2. Настроено CORS-промежуточное ПО для кросс-доменных запросов:

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

3. Основные функциональные модули

Аутентификация:

- Реализовано хеширование паролей с использованием passlib (алгоритм bcrypt)
- Эндпоинт для создания пользователей с валидацией уникальности email

Работа с постами:

- CRUD операции для постов (создание, чтение, обновление, удаление)
- Поддержка тегов и категорий для постов
- Частичное обновление постов (PATCH)
- Поиск по заголовку и содержанию постов

Категории и теги:

- Отдельные CRUD операции для категорий
- Управление тегами постов

Комментарии:

- Создание и получение комментариев
- Привязка комментариев к постам и пользователям

Избранное:

- Добавление постов в избранное
- Просмотр избранного пользователя

4. Особенности реализации

1. Безопасность:

- Пароли хранятся в хешированном виде
- Все операции с БД выполняются в рамках сессий с обработкой исключений
- Реализованы проверки существования связанных сущностей

2. Валидация данных:

- Используются Pydantic-схемы для валидации входных и выходных данных
- Подробные сообщения об ошибках (404 для отсутствующих ресурсов, 400

для конфликтов)

3. Логика работы:

- Автоматическое проставление временных меток (created_at)
- Поддержка частичного обновления сущностей
- Поиск с использованием ILIKE для регистронезависимых запросов

4. Масштабируемость:

- Четкое разделение моделей, схем и эндпоинтов
- Использование dependency injection для работы с БД
- Подготовка к реализации авторизации (заглушка user_id в комментариях)

Сервер предоставляет полноценный API для блог-платформы с возможностями:

- Управления пользовательским контентом (постами, комментариями)
- Организации контента через категории и теги
- Персонализации через механизм избранного
- Поиска по содержимому платформы

Все эндпоинты доступны по префиксу /api, что обеспечивает согласованность с прокси-конфигурацией Nginx.

Для дальнейшего развития системы предусмотрены:

1. Добавление аутентификации и авторизации
2. Реализация пагинации для списковых endpoint'ов
3. Добавление системы рейтингов и лайков
4. Расширение поисковых возможностей
5. Реализация уведомлений и подписок

Этот серверный компонент полностью соответствует REST-архитектуре и предоставляет все необходимые endpoint'ы для работы клиентского приложения.

```
main.py x
src > main.py
1 from fastapi import FastAPI, HTTPException, Depends, Query, Request
2 from fastapi.middleware.cors import CORSMiddleware
3 from sqlalchemy.orm import Session
4 from sqlalchemy import or_, text
5 from typing import List, Optional
6 from datetime import datetime
7 from . import models, schemas
8 from .database import SessionLocal, engine
9 from passlib.context import CryptContext
10
11 # Создаем таблицы при запуске
12 try:
13     # Создаем ENUM тип, если он не существует
14     with engine.connect() as conn:
15         # Проверяем существование типа
16         result = conn.execute(text(
17             "SELECT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'userrole')"
18         )).scalar()
19         if not result:
20             conn.execute(text(
21                 "CREATE TYPE userrole AS ENUM ('ADMIN', 'MODERATOR', 'USER')"
22             ))
23             conn.commit()
24
25     # Создаем таблицы с проверкой существования
26     models.Base.metadata.create_all(bind=engine, checkfirst=True)
27 except Exception as e:
28     print(f"Error during database initialization: {e}")
29     # Продолжаем выполнение, так как таблицы могут уже существовать
30
31 app = FastAPI(
32     title="MAI Backend API",
33     description="API для блог-платформы",
34     version="1.0.0",
35     root_path="/api"
36 )
```

Рисунок 2. Инициализация приложения

```
# Настройка CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Настройка хеширования паролей
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Рисунок 3. Настройка CORS и подключение к БД


```

# Создание пользователя
@app.post("/users", response_model=schemas.User)
async def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    try:
        # Проверяем, существует ли пользователь с таким email
        db_user = db.query(models.User).filter(models.User.email == user.email).first()
        if db_user:
            raise HTTPException(status_code=400, detail="Email already registered")

        # Хешируем пароль
        hashed_password = pwd_context.hash(user.password)

        # Создаем пользователя
        db_user = models.User(
            username=user.username,
            email=user.email,
            full_name=user.full_name,
            hashed_password=hashed_password,
            is_active=True,
            created_at=datetime.utcnow()
        )

        db.add(db_user)
        db.commit()
        db.refresh(db_user)
        return db_user
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=str(e))

```

Рисунок 4. Ручка по созданию пользователя

3 База данных и ORM

3.1 Выбор и настройка базы данных

В качестве системы управления базами данных был выбран PostgreSQL - современная реляционная СУБД с открытым исходным кодом, обладающая следующими преимуществами:

- Надежность и отказоустойчивость
- Поддержка сложных типов данных (включая JSON и пользовательские типы)
- Возможность создания пользовательских ENUM-типов
- Высокая производительность
- Поддержка транзакций и ACID-свойств

Настройка подключения к базе данных осуществляется через переменные окружения, что обеспечивает гибкость при развертывании в разных средах:

```
# database.py
```

```
DB_USER = os.getenv("POSTGRES_USER", "admin")
DB_PASSWORD = os.getenv("POSTGRES_PASSWORD", "admin")
DB_NAME = os.getenv("POSTGRES_DB", "mai_project")
DB_HOST = os.getenv("POSTGRES_HOST", "db")
```

```
SQLALCHEMY_DATABASE_URL =
f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}/{DB_NAME}"
```

3.2 ORM и модели данных

Для работы с базой данных используется SQLAlchemy - наиболее популярный ORM для Python, который предоставляет:

- Декларативный синтаксис для определения моделей
- Поддержку отношений между таблицами
- Гибкую систему сессий и транзакций
- Возможность выполнения сырых SQL-запросов

Структура базы данных включает следующие сущности:

3.2.1 Пользователи (User)

```
class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    username = Column(String(50), unique=True, nullable=False)
    email = Column(String(120), unique=True, nullable=False)
    role = Column(Enum(UserRole), default=UserRole.USER)
    is_active = Column(Boolean, default=True)
    ...
```

Особенности:

- Использование ENUM-типа для ролей пользователей
- Хранение хешированных паролей
- Связи с комментариями и избранным

3.2.2 Категории (Category)

```
class Category(Base):
    __tablename__ = 'categories'

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
```

```
parent_id = Column(Integer, ForeignKey('categories.id'))
```

```
...
```

Особенности:

- Иерархическая структура (подкатегории)
- Связь один-ко-многим с постами

3.2.3 Теги (Tag)

```
class Tag(Base):
```

```
    __tablename__ = 'tags'
```

```
    id = Column(Integer, primary_key=True)
```

```
    name = Column(String(50), unique=True, nullable=False)
```

```
...
```

Особенности:

- Связь многие-ко-многим с постами через ассоциативную таблицу post_tags

3.2.4 Посты (Post)

```
class Post(Base):
```

```
    __tablename__ = 'posts'
```

```
    id = Column(Integer, primary_key=True)
```

```
    title = Column(String(200), nullable=False)
```

```
    content = Column(Text)
```

```
    category_id = Column(Integer, ForeignKey('categories.id'))
```

```
...
```

Особенности:

- Поддержка Markdown-контента (тип Text)
- Связи с категориями, тегами, комментариями и избранным
- Хранение метаданных (источник, изображение)

3.2.5 Комментарии (Comment)

```
class Comment(Base):
```

```
    __tablename__ = 'comments'
```

```
    id = Column(Integer, primary_key=True)
```

```
    content = Column(Text, nullable=False)
```

```
    user_id = Column(Integer, ForeignKey('users.id'))
```

```
    post_id = Column(Integer, ForeignKey('posts.id'))
```

```
...
```

3.2.6 Избранное (Favorite)

```
class Favorite(Base):
```

```
    __tablename__ = 'favorites'
```

```
    id = Column(Integer, primary_key=True)
```

```
    user_id = Column(Integer, ForeignKey('users.id'))
```

```
    post_id = Column(Integer, ForeignKey('posts.id'))
```

```
...
```

3.3 Миграции базы данных

Для инициализации структуры базы данных используется механизм миграций

SQLAlchemy. Файл migrations.py содержит логику создания всех таблиц:

```
def init_db():  
    Base.metadata.create_all(bind=engine)
```

Особенности подхода:

- Автоматическое создание таблиц при старте приложения
- Проверка существования таблиц перед созданием
- Поддержка пользовательских типов (ENUM)

3.4 Схемы данных (Pydantic)

Для валидации и сериализации данных используются Pydantic-схемы, которые обеспечивают:

- Валидацию входных данных API
- Сериализацию выходных данных
- Преобразование между ORM-моделями и JSON

Пример схемы для поста:

```
class PostBase(BaseModel):  
    title: str  
    content: str  
    source_url: Optional[str] = None  
    category_id: int  
    tags: List[int] = []
```

Особенности:

- Отдельные схемы для создания, обновления и чтения сущностей
- Поддержка вложенных моделей (например, пост с категорией и тегами)
- Автоматическая конвертация дат и других сложных типов

Особенности:

- Отдельные схемы для создания, обновления и чтения сущностей
- Поддержка вложенных моделей (например, пост с категорией и тегами)
- Автоматическая конвертация дат и других сложных типов

HTTP Request -> PostCreate schema -> Post model -> DB -> Post schema -> HTTP Response

3.6 Заключение

Реализованная структура базы данных и ORM-слой обеспечивают:

- Четкое разделение моделей данных
- Гибкость при изменении структуры
- Безопасность данных (валидация на всех уровнях)
- Эффективное выполнение запросов
- Поддержку сложных отношений между сущностями

Использование PostgreSQL в сочетании с SQLAlchemy и Pydantic создает надежную основу для развития приложения, позволяя легко добавлять новые функции и масштабировать систему.

4 Контейнеризация

4.1 Общая архитектура решения

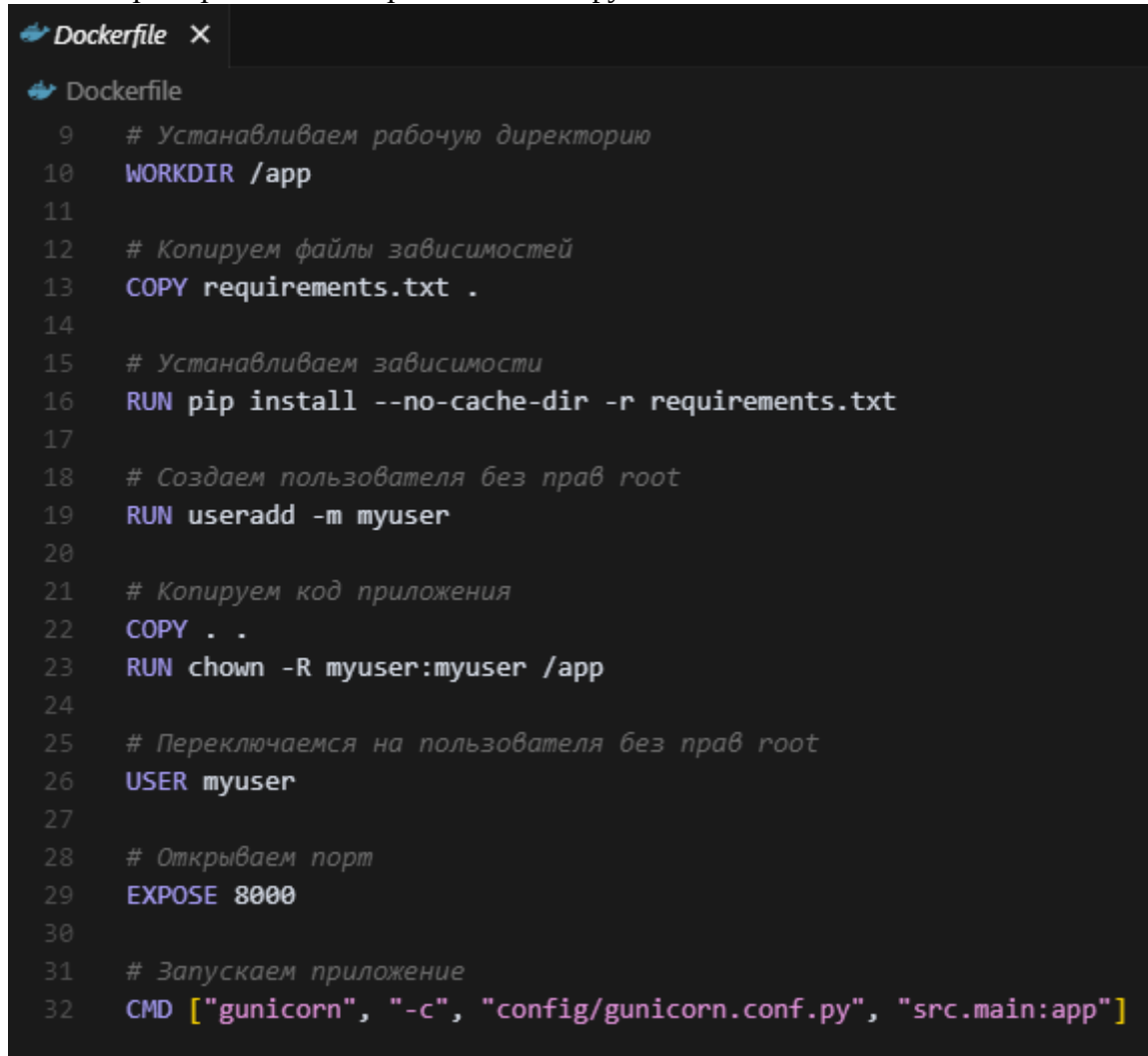
Система разворачивается с использованием Docker [3] и состоит из трех основных сервисов:

1. **Nginx** - веб-сервер и обратный прокси
2. **FastAPI приложение** - основной backend на Python
3. **PostgreSQL** - база данных

Все сервисы объединены в единую сеть app-network для безопасного взаимодействия.

4.2 Dockerfile приложения

Основной образ приложения строится на базе python:3.11-slim и включает:



```
Dockerfile X
Dockerfile
9  # Устанавливаем рабочую директорию
10 WORKDIR /app
11
12 # Копируем файлы зависимостей
13 COPY requirements.txt .
14
15 # Устанавливаем зависимости
16 RUN pip install --no-cache-dir -r requirements.txt
17
18 # Создаем пользователя без прав root
19 RUN useradd -m myuser
20
21 # Копируем код приложения
22 COPY . .
23 RUN chown -R myuser:myuser /app
24
25 # Переключаемся на пользователя без прав root
26 USER myuser
27
28 # Открываем порт
29 EXPOSE 8000
30
31 # Запускаем приложение
32 CMD ["gunicorn", "-c", "config/gunicorn.conf.py", "src.main:app"]
```

Рисунок 5. Dockerfile

Ключевые особенности:

- Минимальный базовый образ (slim-версия Python)
- Установка только необходимых системных пакетов (postgresql-client)
- Принцип минимальных привилегий (не-root пользователь)
- Кэширование слоев для ускорения сборки
- Использование Gunicorn как production-ready сервера

4.3 Docker Compose конфигурация

Файл docker-compose.yml описывает всю инфраструктуру:

```
docker-compose.yml
1  version: '3.8'
2
3  services:
4    nginx:
5      image: nginx:1.24
6      ports:
7        - "80:80"
8      volumes:
9        - ./config/nginx.conf:/etc/nginx/conf.d/default.conf
10       - ./web:/app/web
11       - ./public:/app/public
12      depends_on:
13        - app
14      networks:
15        - app-network
16
17    app:
18      build: .
19      ports:
20        - "8000:8000"
21      environment:
22        - POSTGRES_USER=admin
23        - POSTGRES_PASSWORD=admin
24        - POSTGRES_DB=mai_project
25        - POSTGRES_HOST=db
26      depends_on:
27        db:
28          condition: service_healthy
29      networks:
30        - app-network
31
32    db:
33      image: postgres:15
34      environment:
35        - POSTGRES_USER=admin
36        - POSTGRES_PASSWORD=admin
37        - POSTGRES_DB=mai_project
38      volumes:
39        - postgres_data:/var/lib/postgresql/data
40      healthcheck:
41        test: ["CMD-SHELL", "pg_isready -U admin -d mai_project"]
42        interval: 5s
43        timeout: 5s
44        retries: 5
45      networks:
46        - app-network
47
48    volumes:
49      postgres_data:
50
51    networks:
52      app-network:
53        driver: bridge
```

Рисунок 6. Docker-compose

Особенности конфигурации:

1. **Nginx:**

- Использование официального образа
- Монтирование конфигурации и статических файлов
- Проброс порта 80

2. **Приложение:**

- Локальная сборка из Dockerfile
- Настройки подключения к БД через переменные окружения

- Явная зависимость от здоровой БД
- 3. **PostgreSQL:**
 - Том для постоянного хранения данных
 - Healthcheck для проверки готовности
 - Автоматическое создание БД при первом запуске
- 4. **Сеть:**
 - Изолированная bridge-сеть для безопасного взаимодействия

4.4 Управление через Makefile

Для удобства разработки реализованы основные команды:

```
build:  # Сборка образов
up:     # Запуск всех сервисов
down:   # Остановка
logs:   # Просмотр логов
ps:     # Статус сервисов
clean:  # Полная очистка
test:   # Запуск тестов
init:   # Первоначальная настройка (build + up + migrate)
migrate: # Применение миграций
```

Преимущества такого подхода:

- Стандартизация команд для всех разработчиков
- Автоматизация рутинных операций
- Возможность расширения для CI/CD

4.5 Принципы контейнеризации

В проекте соблюдаются лучшие практики:

1. **Безопасность:**
 - Непривилегированные пользователи в контейнерах
 - Изолированные сетевые пространства
 - Минимальные базовые образы
2. **Надежность:**
 - Healthcheck для БД
 - Явные зависимости между сервисами
 - Тома для постоянного хранения данных
3. **Масштабируемость:**
 - Четкое разделение сервисов
 - Возможность горизонтального масштабирования
 - Гибкая конфигурация через переменные окружения
4. **Повторяемость:**
 - Идемпотентные команды развертывания
 - Версионирование образов
 - Декларативное описание инфраструктуры

4.6 Процесс развертывания

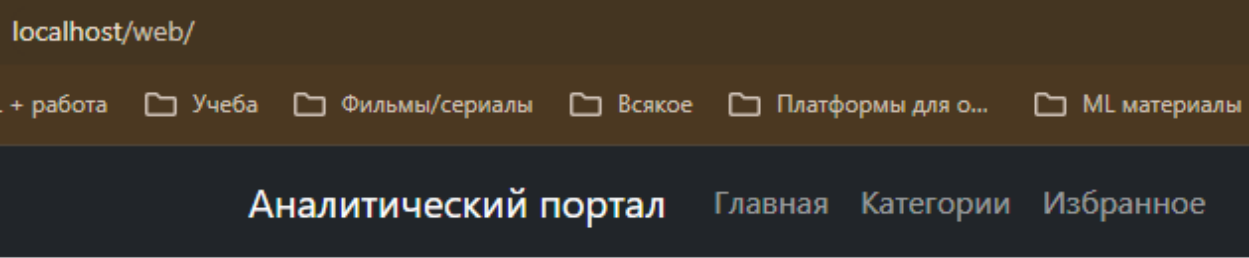
Типичный workflow:

1. Инициализация: `make init`
 - Сборка образов
 - Запуск контейнеров
 - Применение миграций
2. Разработка:
 - Логи: `make logs`

- Тесты: make test
- 3. Остановка: make down или make clean (полная очистка)

5 Демонстрация

Для запуска приложения необходимо прописать make init, после чего соберутся и запустятся все необходимые образы.



Последние статьи

Рисунок 7. Стартовая страница

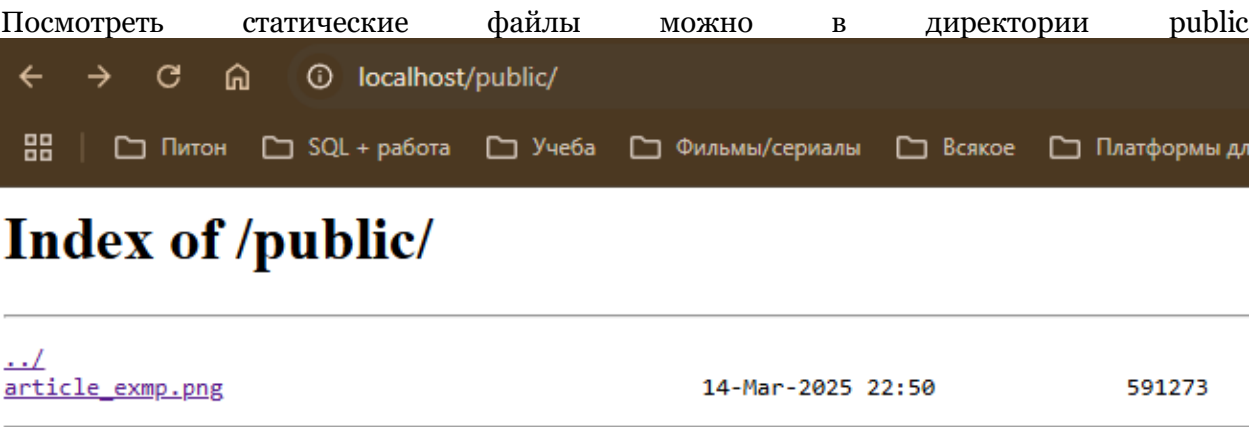


Рисунок 8. Статические файлы

В FastAPI предусмотрена удобная swagger документация доступная «из коробки» по пути /docs здесь можно в удобном UI интерфейсы протестировать все доступные эндпоинты

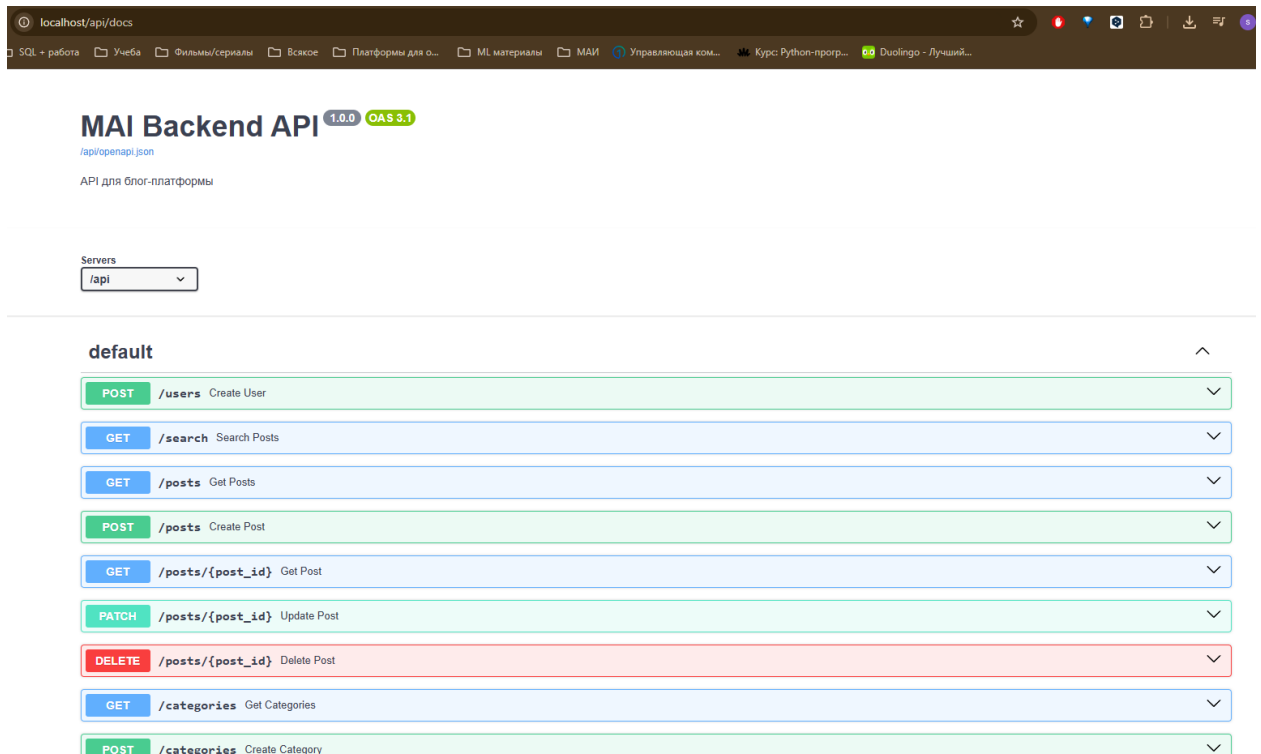


Рисунок 9. Документация

Протестируем функционал создания категории и поста

Создадим категорию через curl -X 'POST' \

'http://localhost/api/categories' \

-H 'accept: application/json' \

-H 'Content-Type: application/json' \

-d '{

"name": "Root Category",

"description": "Root category"

}'

Проверим ее наличие

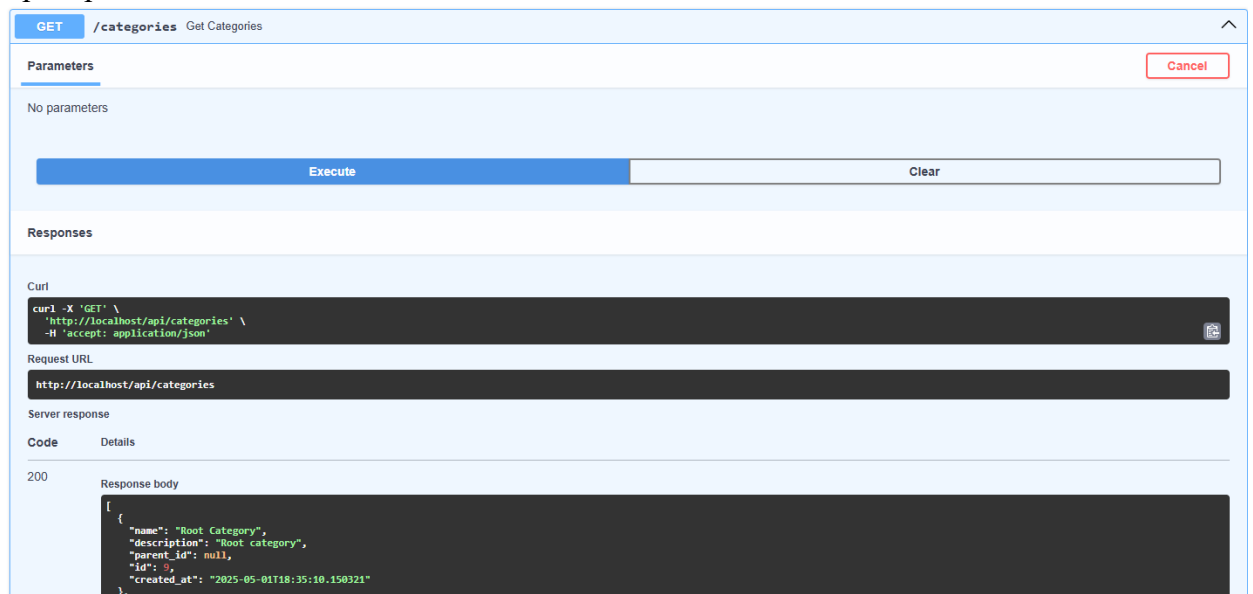


Рисунок 10. Получение списка категорий

Теперь создадим пост ссылаясь на эту категорию через интерфейс

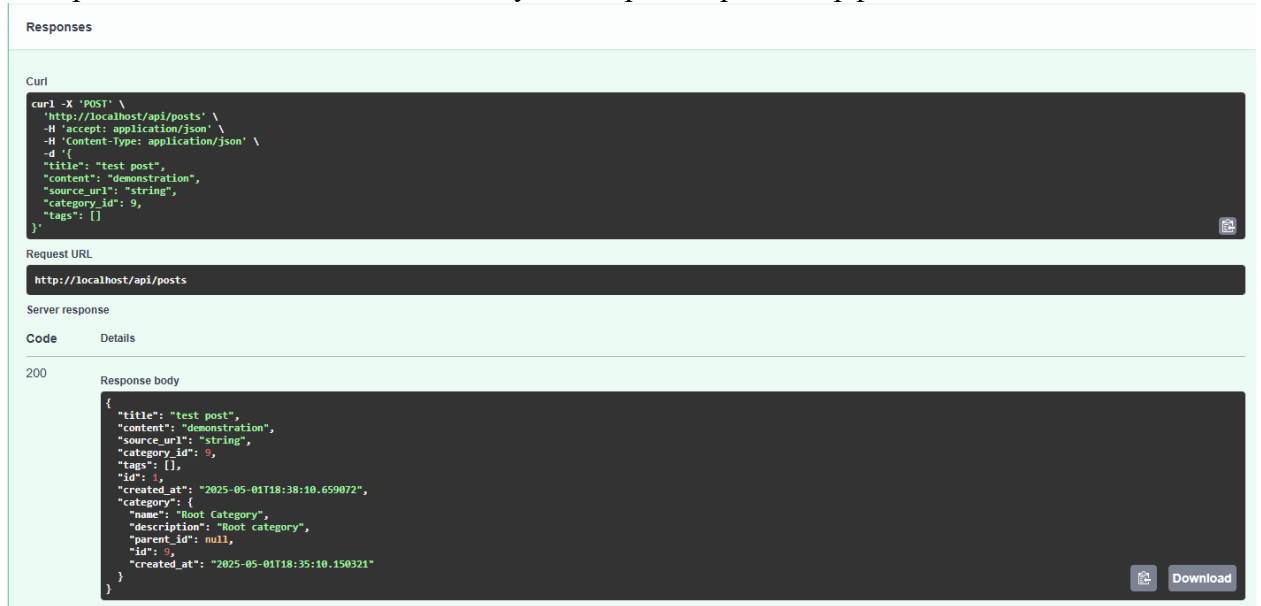


Рисунок 11. Создание поста

Теперь на главной странице мы видим появившийся пост

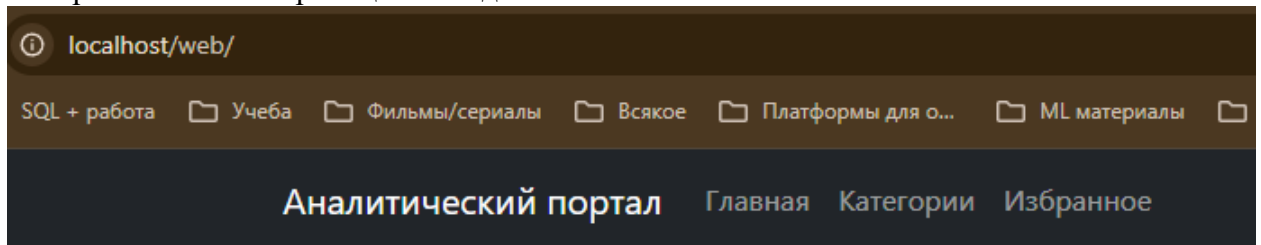


Рисунок 12. Главная страница с постом

Также можно проверить список постов через url

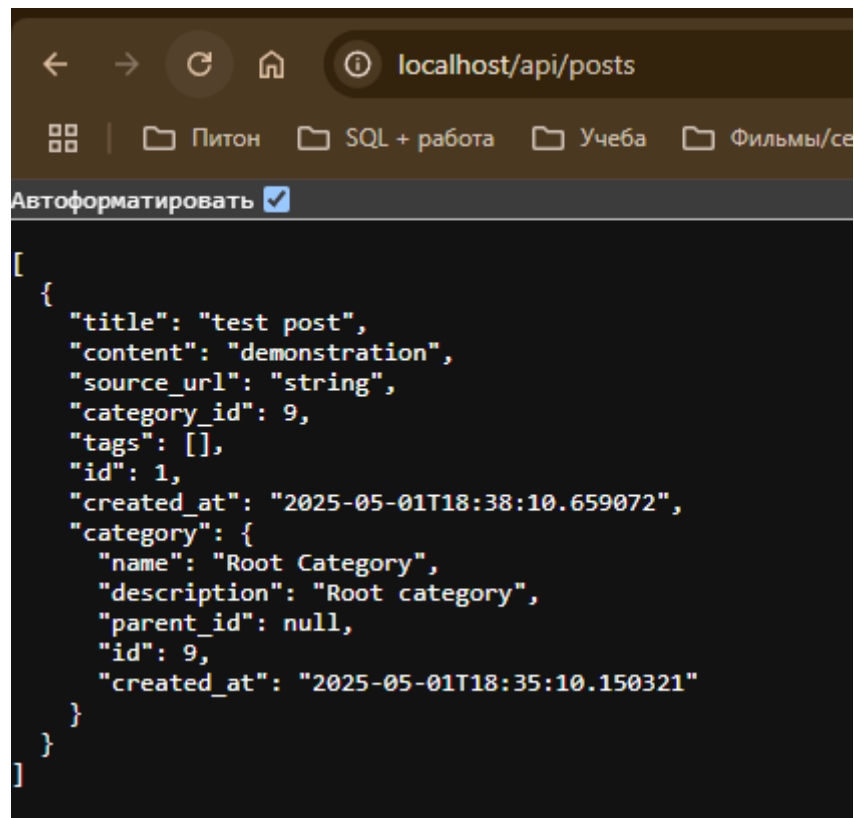


Рисунок 13. Список постов

6 Выводы

В ходе выполнения курсовой работы была разработана полнофункциональная блог-платформа с использованием современных технологий и подходов в веб-разработке. Работа демонстрирует комплексное решение, включающее backend на FastAPI, фронтенд (SPA), базу данных PostgreSQL и контейнеризацию с помощью Docker.

Ключевые достижения:

1. Архитектура системы:

- Реализована четкая трехзвенная архитектура (клиент — сервер — БД)
- Применен принцип разделения ответственности между компонентами
- Обеспечена масштабируемость каждого компонента системы

2. Backend-разработка:

- Использован FastAPI как современный высокопроизводительный фреймворк
- Реализовано RESTful API с полным набором CRUD операций
- Настроена система валидации и сериализации данных через Pydantic
- Обеспечена безопасность (хеширование паролей, CORS)

3. Работа с данными:

- Выстроена реляционная модель данных в PostgreSQL
- Реализованы сложные связи между сущностями (многие-ко-многим, иерархии)
- Применен SQLAlchemy как мощный и гибкий ORM
- Настроены автоматические миграции базы данных

4. Инфраструктура:

- Полноценная контейнеризация приложения (Docker)
- Автоматизация развертывания через Docker Compose
- Настроен Nginx как обратный прокси и для раздачи статики
- Реализованы health checks для мониторинга состояния сервисов

Знания, полученные в данной курсовой работе, определенно помогут мне в дальнейшем, я освоил множество новых для себя инструментов и подходов для разработки качественного бэкенд приложения.

Список литературы

- [1] Руководство для начинающих. URL: https://nginx.org/ru/docs/beginners_guide.html (дата обращения: 15.03.2025)
- [2] FastAPI. URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.03.2025)
- [3] Dockerfile reference. URL: <https://docs.docker.com/reference/dockerfile/> (дата обращения: 12.04.2025)